CS310 - Mobile Application Development Project Step 1: Proposal & Setup

(GitHub Repository: https://github.com/mesely/ChefInPocket.git)

1. Project Title and Short Description

Project Title: ChefInPocket

Short Description: ChefInPocket is a clever mobile cooking companion. It helps users find meals based on cuisine, discover recipes using the ingredients they already possess, and adjust serving sizes on the fly. The app also features a smart grocery list.

2. Group Members

Mehmet Selman Yılmaz - 31158 - RAG Architect & Data Scientist | Learning & Research Lead
Cem Özkul - 34183 - Backend Developer & Data Engineer | Testing & Quality Assurance Lead
Emir Keskin - 31020 - Backend Lead | Integration & Repository Lead
Şemse Doğa Atılğan - 34450 - Frontend Developer & Data Analyst | Presentation & Communication Lead Nilsu Saraçlar - 34210 - Frontend Lead (UI/UX) | Documentation & Submission Lead
Bora Demirkol - 32361 - Full Stack Developer | Project Coordinator

3. Main Purpose & The Problem It Solves

The Problem: Many people open their fridge, see a bunch of ingredients, and still have no idea what to cook. Most recipe apps or cookbooks suggest meals that require items they don't have, which leads to extra grocery shopping, wasted food, and frustration. As a result, users spend more time searching for recipes than actually cooking.

Main Purpose: ChefInPocket is designed to make cooking easier and more practical. Instead of asking users to fit into recipes, the app adapts recipes to what users already have at home. By simply selecting available ingredients, users can quickly discover meals that match their kitchen. This approach helps save time, reduce food waste, and makes cooking more enjoyable and accessible.

4. Target Audience

ChefInPocket targets a wide range of audience such as students, busy professionals, homemakers and people who want to share their own recipes with others. It helps users to decide on what to cook with ingredients they already have, discover new meals and connect through shared recipes which makes cooking easier, faster and accessible for everyone.

5. Core and Optional Features

5.1.1.Stateful RAG-Powered Culinary AI:

Uses dense vector embeddings to find recipes based on what the user has in their kitchen. The NLP assistant uses a continuous Retrieval-Augmented Generation (RAG) pipeline with session memory to rewrite recipes on the fly to replace missing ingredients. It also handles follow-up contextual changes (like "Make this spicier") without losing the conversation state.(see Appendix D, Figure 4 for the detailed pipeline flow).

5.1.2. Ingredient-Based Recipe Filtering System with Personalization: Users can select the ingredients they currently have, and the app will suggest recipes that best match those items.

Over time, the system learns user preferences such as favorite cuisines or dietary habits, making the recommendations more relevant and personalized

5.1.3.Dynamic Serving & Ingredient Scaling with Clean UI Experience: The app allows users to easily adjust the number of servings, and all ingredient quantities update automatically. This feature is supported by a simple and clean interface, making it easy to use even while cooking without causing confusion or distraction.

5.1.4.User Recipe Sharing & Secure Data Management: Users can save their favorite recipes and share their own creations with others. At the same time, the app is designed with data privacy and security in mind. It follows good software engineering practices such as modular design and scalability to ensure a reliable and maintainable system.

Optional "Nice-to-have" Features (If time permits):

5.2.1.AI "Chef Personas" for Cultural Localization: Users can utilize personalized models tailored to different cuisines (such as Italian or Turkish). For example, when Turkish cuisine mode is selected, the tiramisu recipe will be presented using labneh, a more readily available cheese, instead of mascarpone.

5.2.2.Minimalist & Personalized User Experience: The interface is designed to be simple, clean, and easy to navigate. As users interact with the app, it adapts to their habits by highlighting frequently used features, preferred cuisines, and saved recipes, creating a smoother and more personalized experience.

6. Platform: The application will be developed using Flutter for mobile.

7. Data Storage

Database Choice: MongoDB (NoSQL)

Data types: collections (users, ingredients, recipes, chat sessions, messages) will keep profiles, inventories, and NLP vectors etc.. Documents use id fields as primary keys and ObjectIds to link to other documents. The proposed schema and relationships are illustrated in Appendix B (Figure 2). But this schema is still meant to be flexible.

8. Unique Selling Point

Most recipe apps just hand you a static list of instructions and expect you to have every ingredient. ChefInPocket is different because it adapts to you. Our biggest standout is the RAG-powered AI that actually rewrites recipes on the fly based on whatever is available in your fridge. It has an automatic portion scaling, and you get an app that feels like a real kitchen assistant rather than just a digital cookbook. (Refer to Appendix A for the high-fidelity UI mockups)

9. Challenges

Integrating the RAG-based AI recipe system with the Flutter mobile application while preserving quick response times will be one of the primary difficulties. It can be difficult to continually oversee conversational state and ingredient substitutions. Careful system architecture (outlined in Appendix C, Figure 3) will also be needed to handle dynamic UI updates for ingredient filtering and recipe scalability, as well as to build an effective MongoDB schema.

Appendix

A.

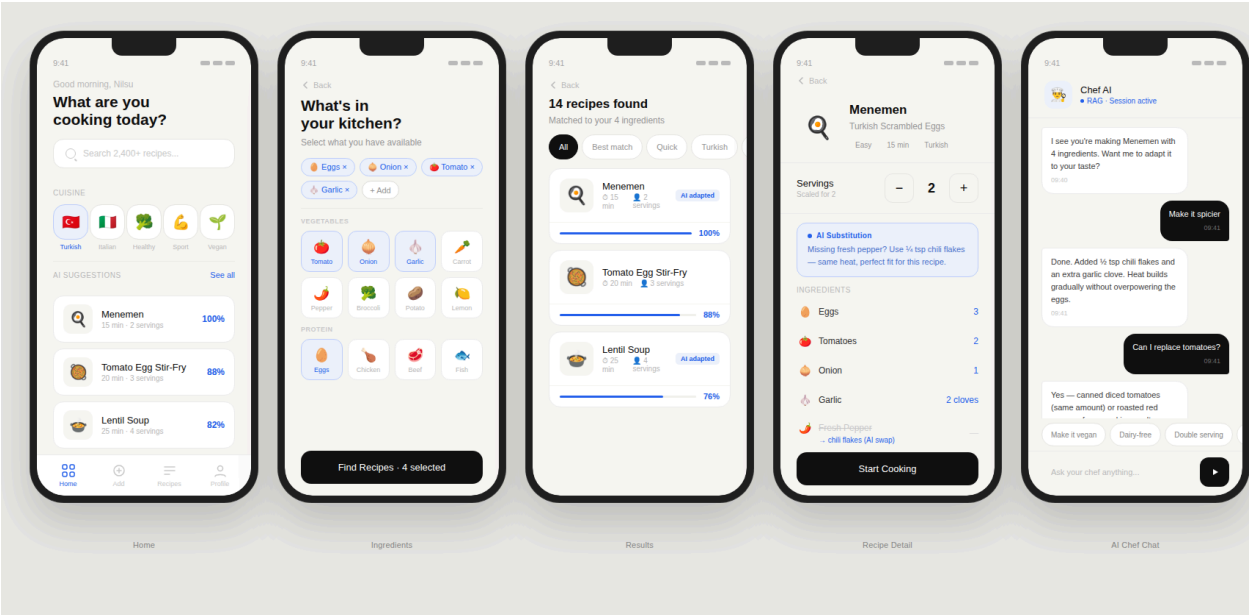

Figure 1: High-fidelity Figma mockups of screens

B.

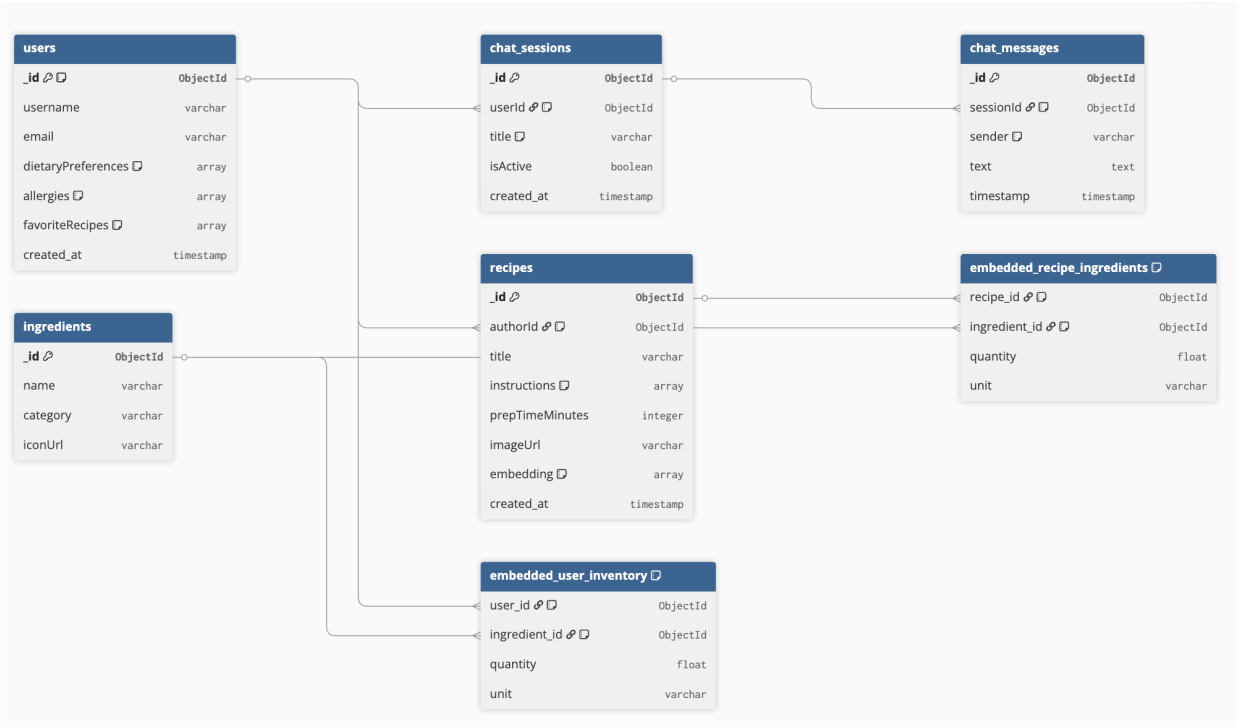

Figure 2: Proposed MongoDB Document Schema and Relational Architecture for ChefInPocket

C.

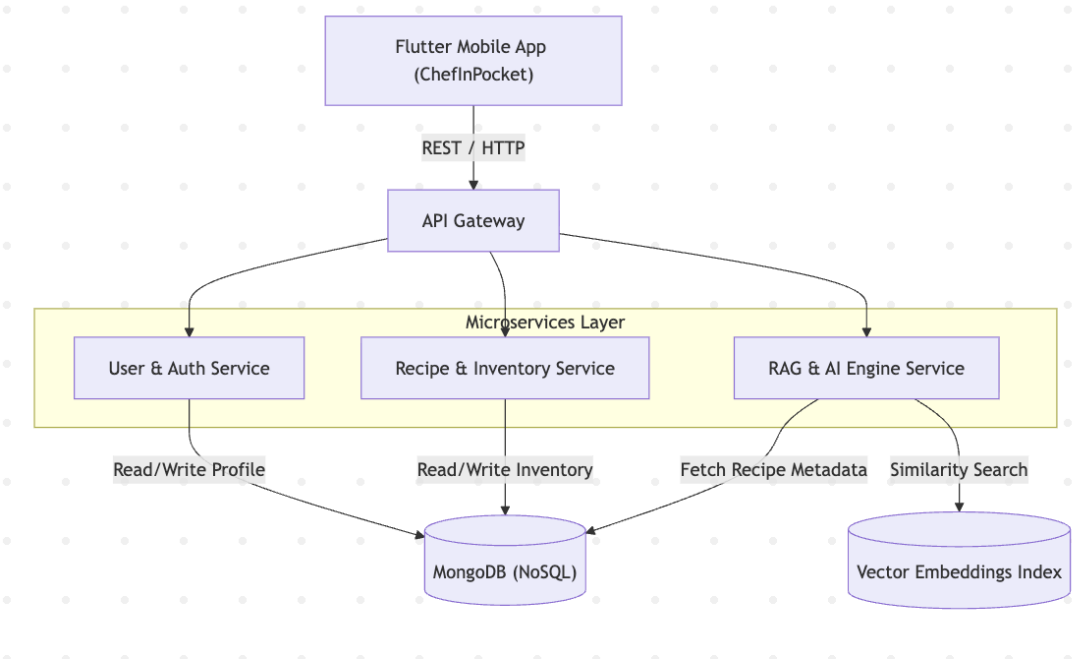

Figure 3: High-Level Microservices Architecture

D.

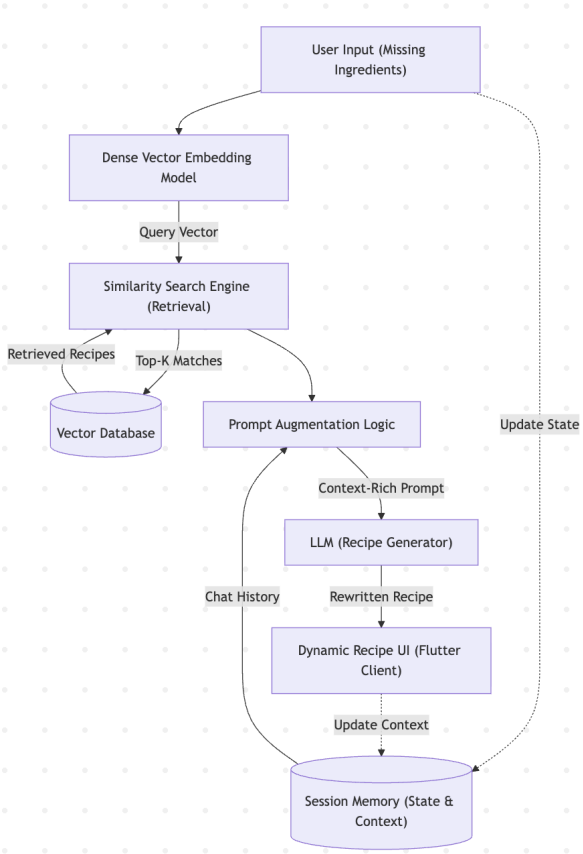

Figure 4: Stateful RAG Pipeline Flow